Usando o Claude com o DevFlow

Há duas formas de trabalhar com o Claude aqui, e elas servem a coisas diferentes:

- **O painel do Claude**, dentro do app — para perguntar sobre as suas requests, montar fluxos e rodar chamadas sem sair da tela.
- **O Claude Code no terminal**, conectado ao DevFlow por um servidor MCP — para quando o trabalho é no código, mas você quer que o agente dispare as requests que já estão configuradas aqui.

As duas usam a **mesma CLI** (`claude`) por baixo. O que muda é quem a invoca e o que ela enxerga.

1. O painel do Claude

Onde fica

Ícone do Claude na barra lateral. O painel abre ao lado, com o histórico da conversa.

Entrando

Clique no ícone de chave 🔑 no topo do painel (ou no botão **Fix sign-in / API key**, que aparece junto de um erro de credencial). Há três caminhos:

Opção	O que usa	Quando faz sentido
Claude subscription	A sua assinatura do Claude	O caso normal. Não consome créditos de API.
Anthropic Console	Créditos de API, cobrados por uso	Se você trabalha com chave de organização
API key	Uma chave colada no campo	Chave avulsa, sem passar pelo login

Os dois primeiros abrem **uma janela de terminal de verdade** rodando `claude auth login`. Isso é de propósito: o fluxo é interativo — abre o navegador e pode pedir que você cole um código de volta. Rodando escondido, ele ficaria travado esperando uma entrada que ninguém consegue digitar.

O perfil separado (importante)

Por padrão, a caixa **"Keep a separate login for DevFlow"** vem marcada. Com ela, o app aponta a CLI para um diretório de configuração próprio:

```
%APPDATA%\com.DevFlow.dev\claude\
```

em vez do `~/.claude` global. O motivo: logar pelo painel não pode derrubar a conta que você já usa no terminal. Com perfis separados, dá para usar a conta pessoal aqui e manter a outra lá fora.

O efeito colateral que confunde: se você já está logado no `claude` do terminal, o painel ainda vai dizer **"not logged in"** — e corretamente, porque está olhando outro diretório. Para reaproveitar aquela conta, **desmarque** a caixa. Aí o perfil passa a enxergar também os seus settings globais, servidores MCP e `CLAUDE.md`.

Para conferir se o login funcionou, o arquivo `.claude.json` do perfil passa a ter `oauthAccount`, `emailAddress` e `organizationName`. A credencial em si fica no `.credentials.json` ao lado — esse não deve ser copiado para lugar nenhum.

A credencial é **por máquina e por pessoa**. Não há nada que você provisione para um colega: ele precisa da assinatura dele, na máquina dele.

O que o painel enxerga

Quando o servidor MCP está ligado (ver seção 2), o painel **passa a configuração dele automaticamente** para a CLI. Você não precisa rodar `claude mcp add` para usar o painel — ele já abre com as ferramentas do DevFlow disponíveis, limitadas ao escopo `mcp__devflow`.

Também dá para definir uma **pasta de trabalho**: é o diretório onde a CLI roda, e portanto o que ela consegue ler. Sem isso, o agente responde sobre as suas collections, mas não sobre o seu código.

Um limite que vale conhecer

O painel roda a CLI em modo `dontAsk`: **não existe prompt de permissão**. Nada pode ficar esperando alguém responder dentro de um painel de chat, então o que não está pré-aprovado é negado e a execução segue. Na prática, o agente do painel faz o que as ferramentas do DevFlow permitem, e não mais que isso.

2. O Claude Code no terminal

Aqui o Claude roda fora do app, e o DevFlow entra como **servidor MCP**: a CLI ganha ferramentas para listar e executar as suas requests.

Ligando o servidor

Settings → Claude Code (MCP) → Turn on.

O servidor escuta **só em 127.0.0.1**, e **toda chamada exige o token** mostrado no painel. Isso não é formalidade: enquanto está ligado, quem tiver o token pode disparar as suas requests com as credenciais dos seus environments. Desligue quando não estiver usando.

Conectando

O painel mostra o comando pronto. Rode **uma vez** no terminal:

```
claude mcp add --transport http devflow http://127.0.0.1:60401/mcp --header "Authorization: Bearer SEU
```

A porta é sorteada pelo sistema na primeira vez e depois **fica guardada**, justamente para que esse comando continue valendo entre reinícios.

> **Se a porta mudar:** o app avisa no painel e pede para rodar o comando de novo. Isso acontece no Windows porque o Hyper-V/WSL reserva faixas de portas altas e as sorteia de novo a cada boot — uma porta que o próprio sistema entregou ontem pode ser recusada hoje. Quando isso ocorre, o DevFlow cai para outra porta em vez de simplesmente não ligar.

Regenerar o token invalida o que você já configurou: quem estava conectado passa a receber 401 até rodar o comando novamente. É esse o propósito do botão.

O token

Fica no armazenamento seguro do sistema (o mesmo do token do GitHub), não em `localStorage`. Ele autoriza disparar qualquer request salva com os seus environments de verdade, então vale pelo menos o mesmo cuidado de uma senha.

3. O que o agente consegue fazer

As ferramentas são as mesmas nos dois modos.

Ler a árvore `list_requests` · `list_folders` · `get_request` · `search_requests` · `list_examples` · `get_example` · `list_environments`

Executar `run_request` · `run_collection` · `run_load_test` · `list_load_tests` · `get_oauth2_token`

Ler o que está na tela `get_active_request` · `get_response` · `get_console_logs`

Escrever `create_folder` · `create_request` · `duplicate_request` · `rename_item` · `set_request_script`

Referência `get_script_api` — a API de scripts (`pm.*`) disponível. Vale chamar antes de pedir que ele escreva um script.

Execução é a mesma do botão Send

`run_request` e `run_collection` passam exatamente pelo mesmo caminho do botão Send: mesma resolução de `{{variáveis}}`, mesmo auth, mesma sanitização de body, mesmos scripts. O que o agente dispara não é uma simulação.

Segredos saem mascarados

`get_request` **redige** o que parece credencial: token, senha, `client_secret` do OAuth2, e headers cujo nome sugira segredo. Referência a variável (`{{token}}`) é preservada de propósito — ela não é segredo, e é justamente o que o agente precisa ver para entender de onde o valor sai.

Por isso existe o **`duplicate_request`**: copiar uma request com OAuth2 lendo e recriando produziria uma cópia sem as credenciais, silenciosamente quebrada. A duplicação acontece dentro do app, onde os valores reais estão, e eles nunca passam pelo modelo.

O que ele não faz

Nada sobe para o GitHub. Criar, renomear ou editar via agente grava **só local** — o push continua sendo ação explícita sua. Isso é deliberado: um agente iterando sobre scripts geraria uma enxurrada de commits.

4. Qual usar quando

Situação	Use
"Por que essa request está dando 400?"	Painel
"Monta um fluxo de cancelamento com reembolso"	Painel
"Roda a collection e me diz o que quebrou"	Qualquer um
Está mexendo no código e precisa bater numa API	Terminal
Quer que o agente leia o repositório inteiro	Terminal

A regra prática: **se o trabalho é sobre as requests, use o painel; se é sobre o código, use o terminal** e deixe o DevFlow como fonte das chamadas.

5. Quando algo dá errado

"Not logged in · Please run /login" no painel A CLI está instalada e rodou — só falta credencial *naquele perfil*. Veja a seção do perfil separado: na maioria das vezes é a caixa "Keep a separate login for DevFlow" marcada enquanto o seu login está no perfil global.

"Credit balance is too low" Você entrou pelo **Anthropic Console**, que cobra por uso de API. Para assinatura, saia e entre por **Claude subscription**.

"Invalid API key" mesmo com a conta configurada Alguma `ANTHROPIC_API_KEY` no ambiente da máquina está vencendo a conta logada — a CLI dá preferência a ela. O app remove essa variável no modo conta, mas vale conferir se ela existe no seu ambiente.

`Failed to bind 127.0.0.1:NNNNN` (erro 10013) Porta **reservada** pelo sistema, não ocupada. Para ver as faixas:

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

O app cai sozinho para outra porta; depois disso, rode o `claude mcp add` novamente com a porta nova.

O agente diz que uma ferramenta não existe A lista acima é o que existe. Se faltar alguma operação, ela precisa ser adicionada em `src/mcp/mcpTools.ts` — não há como o agente contornar.